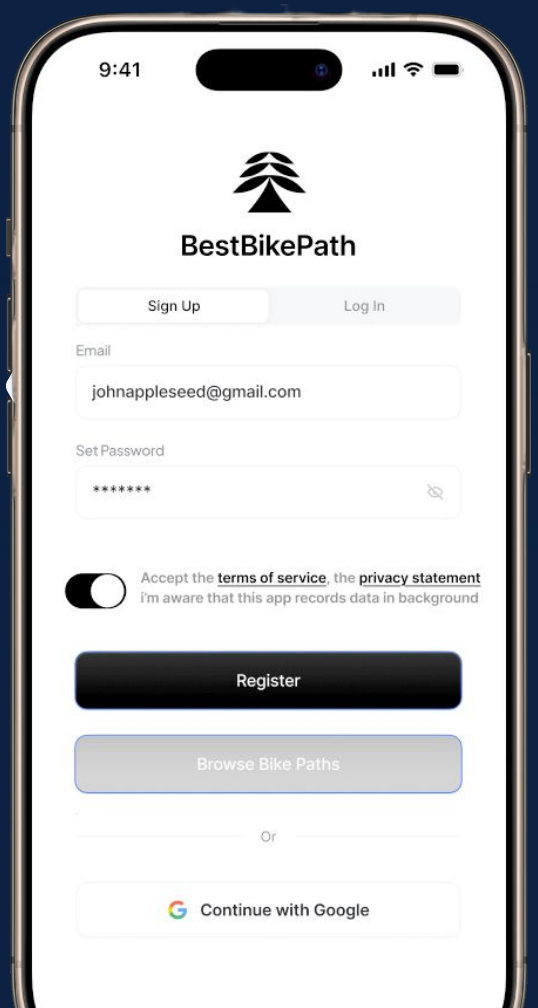
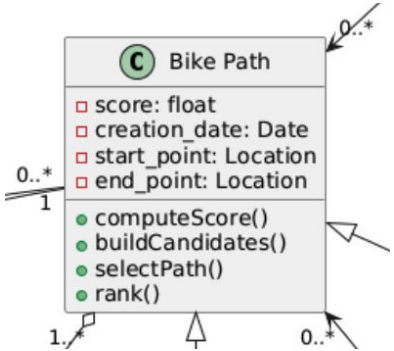
BEST BIKE PATHS.

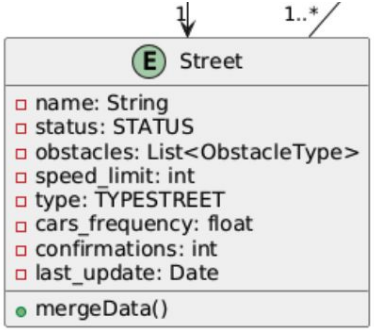


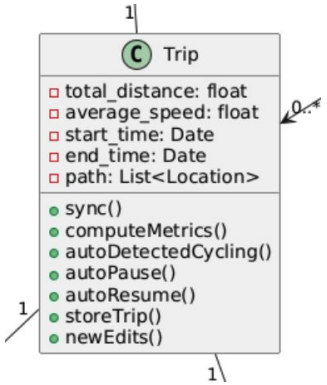
SIMONE CONTROGUERRA ▪ MATTEO DELOGU ▪ GIANLUCA DI PAOLA

MODELING FUNDATIONS









BIKE PATH

A Bike Path is a **public route for cyclists**, either manually defined by users or automatically composed by the system. Formally, it is a set made of:

- a Starting Point;
- a Finishing Point;
- an ordered sequence of traversable **Streets**.

STREET

A Street is a **road segment considered suitable for cycling** (e.g., roadway width, traffic volume, bike lanes)

- It carries attributes such as status and irregularities (e.g., potholes).
- Suitability is determined thanks to the **Navigation System API**.

TRIP

A Trip is a **private, recorded instance** of a Registered User moving from A to B, enriched with sensor data, and (potentially) weather information (provided by the **Weather-Forecast API**).

- It differs from a BP, serving more as a **training record** and offering a more complete experience to the final user.

Unregistered Users

Can only **query the catalog** by specifying Origin and Destination and then view all available bike paths, ordered by overall score; they do not contribute data nor access personal trip/bike path features.

Registered Users can also:

1. **Record personal trips** (with automatic start/stop mechanism, offline capture), enriched with metrics and, optionally, with weather.
2. **Create bike paths** manually by specifying start, finish and the ordered street sequence, with perceived status/irregularities.
3. **Review and validate** obstacles detected from automatic recordings before publication (confirm/correct).
4. **Publish approved content** to the shared catalog: the system verifies street compliance, then merges contributions at street level and propagates updates to affected paths.
5. **Manage account** settings (consents, profile, deletion).

DOMAIN CONTEXT



[A1]

The system assumes **responsible user behavior**: cyclists are expected to comply with guidelines and provide accurate, up-to-date information when contributing, otherwise the shared catalog quality would degrade.

[A2, A3]

BBP assumes identity **feasibility for registered usage**. To access the service, a Registered User:

- must have valid email **credentials**;
- must own a device equipped with GPS, accelerometer and gyroscope, capable of internet communication and computation.

Measurement accuracy is fundamental:

- the sensors of the device must be functioning, **calibrated**, and sufficiently **accurate**;
- **GPS** accuracy must be within acceptable error to detect origin/destination;
- **IMU** sensitivity must be adequate to detect potholes and other irregularities.

[A4, A5, A6]

Runtime dependencies must be considered.

- A stable **internet connection** is required from all users, to perform certain actions.
- The **external service APIs** (Weather-Forecast, Navigation-System) must be implemented, available, and functioning correctly.

[D1, D2, D3]

THE WHOLE CONCEPT OF BBP IS BASED IN CORE REQUIREMENTS

USERS CAN ASK FOR

R6 *The system shall accept user **input for origin and destination** and display all possible bike paths, ordered by score.*

THE SYSTEM OFFERS

R12 *The system shall **record personal trips** and present statistics (distance, time, average speed) and, when available, weather data associated with the trip period.*

R14 *The system shall be able to automatically **create bike paths** (i.e., "Automatic Mode") by processing data from the sensors of the device of the user (GPS, accelerometer, gyroscope).*

THE SYSTEM GUARANTEES

R19 *The system shall **merge multiple contributions** related to the same streets using at least freshness and number of confirmations, propagating the update to bike paths that include them.*

R17 *The system shall **verify the compliance of streets**, allowing streets that satisfy defined criteria to be included in bike paths (e.g., average bike speed VS road limit speed, roadway width, traffic volume, dedicated bike lanes).*

R18 *The system shall **prevent** registered users from publishing bike paths with unallowed streets.*


R A D G

Goals

G1 Enable the cycling community to **access a shared catalog** of safe and reliable **bike paths**.

G2 Allow registered users to **store trips** with automatically-collected sensor data and, if available, associated weather.

G3 **Develop and maintain a collaborative inventory** of bike paths and their score, along with status and obstacle information regarding the streets they are composed of.

G4 Enable **registered users to contribute** to the enrichment of the bike paths catalog, by publishing **automatically-recorded** bike paths.

G5 Enable **registered users to contribute** to the enrichment of the bike paths catalog, by publishing **manually-entered** bike paths.

Goals

G6 Allow **any user** (registered or unregistered) to **search for bike paths** from Origin to Destination, ordered by their effectiveness for such parameters.

G7 Ensure bike paths consist only of **roads suitable and safe** for cycling.

G8 **Reduce errors and false positives** through user confirmation and information merging.

G9 Support a **secure account lifecycle** for registered users, including authentication, recovery, consent capture, and preferences management.


BOUNDARIES

SCENARIO #2: TRIP RECORDING

Boundary takeaway: the inputs are shared world phenomena (session and raw sensing), while the core intelligence (cycling detection and processing) sits inside the machine.

The **user starts cycling**.

The system either **detects such activity** (the system infers a cycling state based on sensor patterns) or is **alerted manually** by the user.

Then, it **starts collecting data** from the device's sensors (GPS, accelerometer, gyroscope), in order to obtain **performance metrics** (e.g., average speed during the trip); the recording of data proceeds also in the case of network disruptions.

When the user is finished, the system **stops the recording, calculates trip metrics** (e.g., distance, duration, average speed) and **shows a summary**.

P2 *A registered user starts cycling.*
[World controlled, not Shared]

P5 *A registered user starts a trip session.*
[World controlled, Shared]

P20 *The system detects when the registered user is cycling and processes relevant data, to obtain performance metrics, to detect obstacles and such.*
[Machine controlled, not Shared]

P12 *The system visually notifies the use that the trip recording has started.*
[Machine controlled, Shared]

P9 *Device sensors (GPS, accelerometer, gyroscope) acquire data.*
[World controlled, Shared]

P13 *The system provides all recorded trip metrics.*
[Machine controlled, Shared]

SCENARIO #3: BIKE PATH MANUAL MODE

Boundary takeaway: user-generated paths are not directly trusted. The machine-controlled safety gate mechanism (API call with prevention) separates proposal (world) from accepted publication (machine-mediated).

A registered user **creates a new bike path** by entering start, finish, and the ordered streets. For each street, they **specify the status** (i.e., the overall condition) and **irregularities** (e.g., potholes).

P6

A registered user manually defines (in terms of streets, status, etc.) and publishes a bike path.
[World controlled, Shared]

The **system validates streets** (checking for compliance) and **stores** the new path as public data.

P16

The system prevents registered users from publishing bike paths with unallowed streets.
[Machine controlled, Shared]

P23

The system stores a new bike path.
[Machine controlled, not Shared]

SCENARIO #6: BIKE PATH QUERY

Boundary takeaway: the user sees only ranked results (shared output), while the algorithmic score computation (and recomputation) remains fully inside the machine.

A user of any kind (registered or not) **specifies Origin and Destination**.

P8 *Any user (registered or not) requires a bike path by specifying origin and destination.*
[World controlled, Shared]

The system **retrieves** all available bike paths, **ranks** them by overall score and shows them to the user, ordered by efficiency.

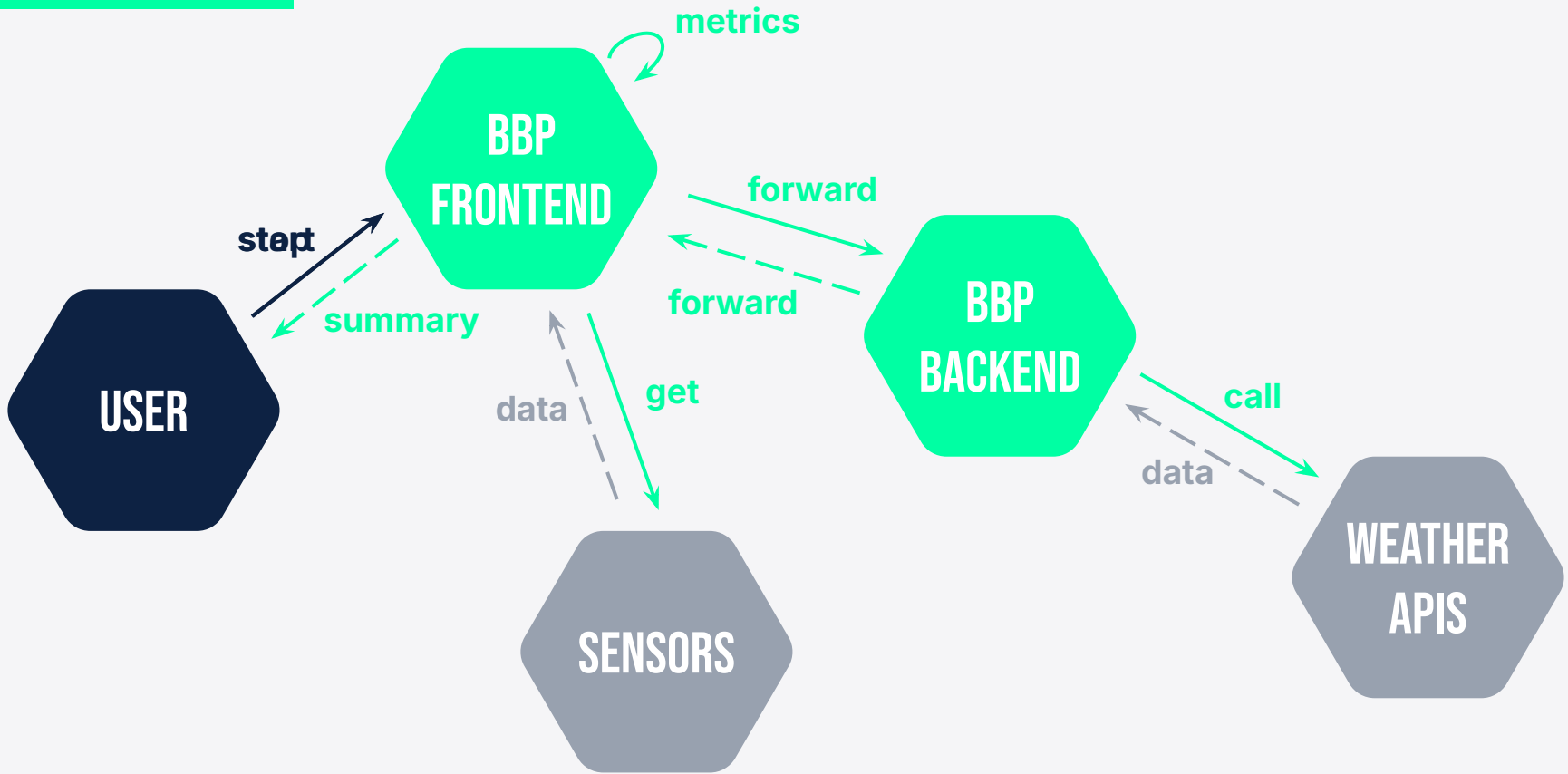
P21 *The system calculates the overall score of a bike path, depending on its road status and e!ectiveness (in terms of allowing any user to reach the desired destination from a given origin).*
[Machine controlled, not Shared]

P15 *The system shows to any user (registered or not) all the possible bike paths, ordered by overall score, to accomplish their request.*
[Machine controlled, Shared]

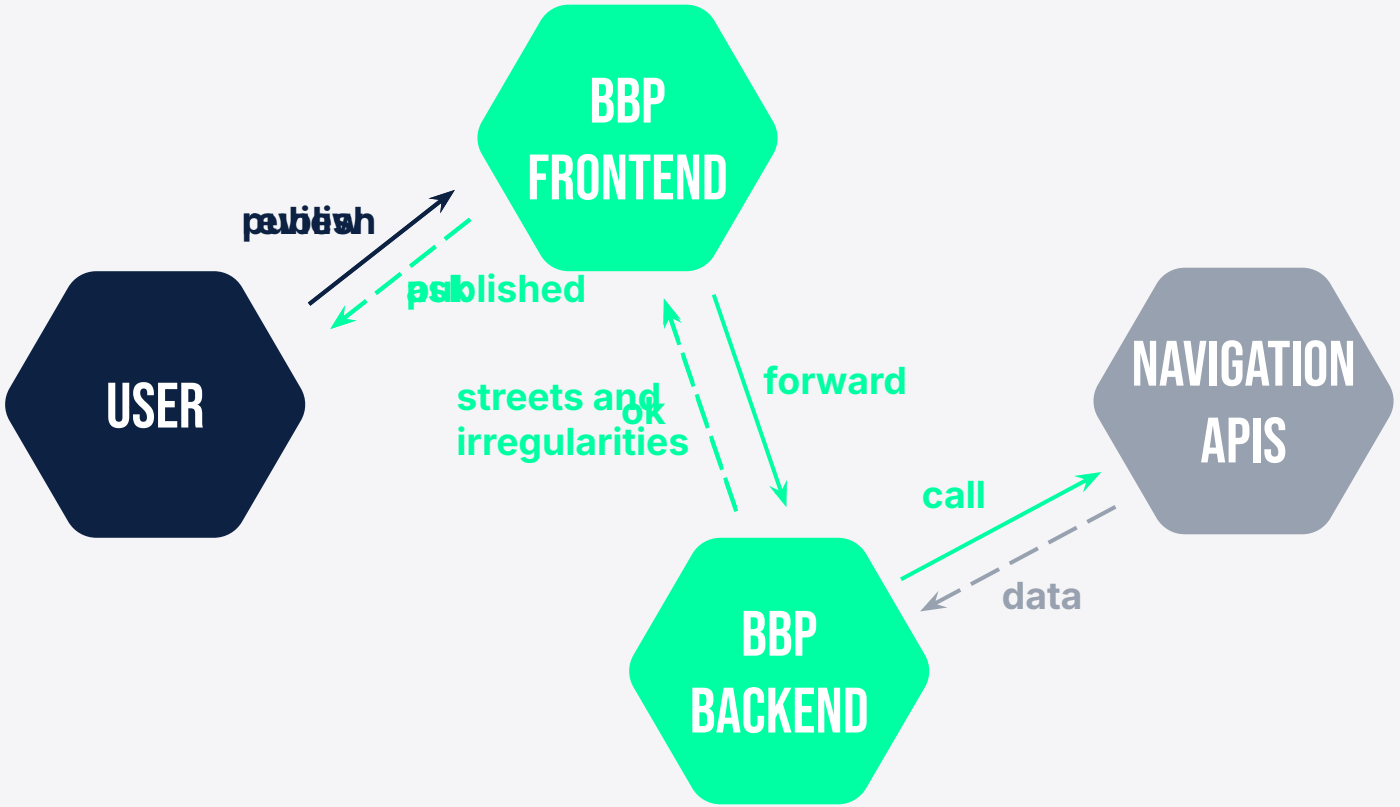
USE CASES



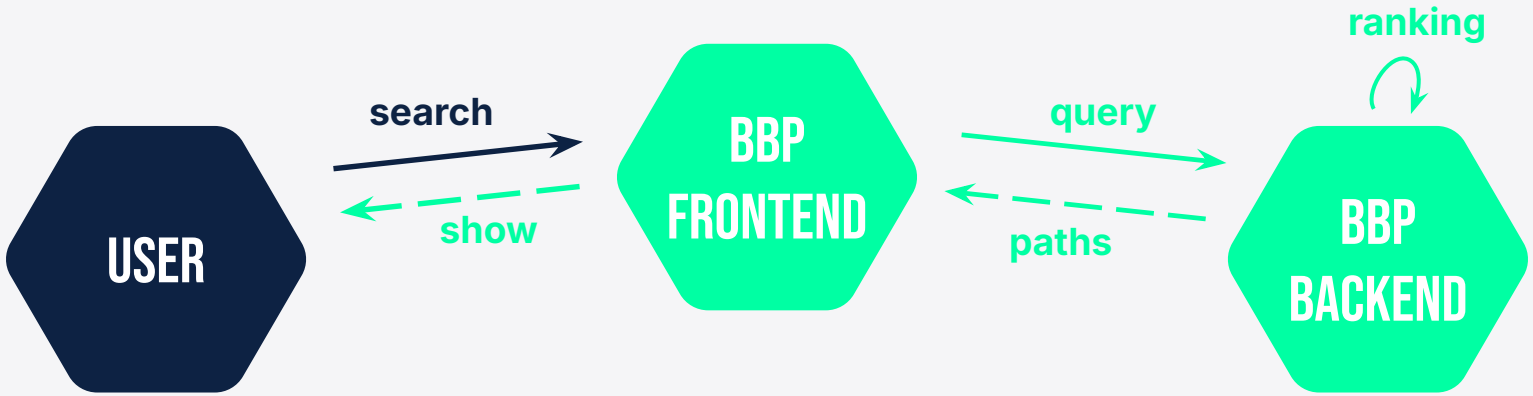
USE CASE #4: RECORDING A TRIP



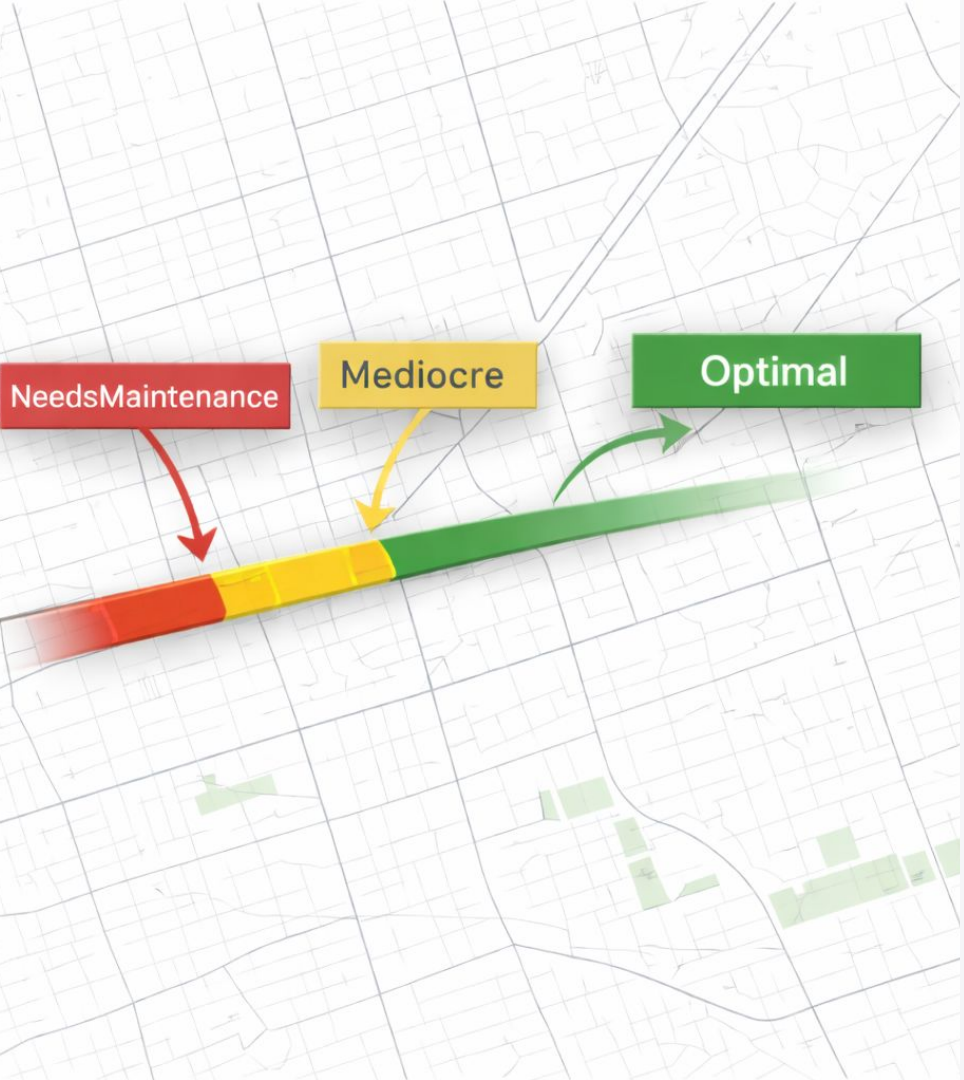
USE CASE #6 + #7: PUBLISHING A BIKE PATH



USE CASE #8:
LOOKING FOR A BIKE PATH




ALLOY CORE



Merging Street Status

`computedStatus[s: Street]: Status`

1. **Freshness aware**
   Only consider the most recent reports
   `mostRecentReports[s]`

2. **Majority based**
   Select the status with most votes between recent reports

3. **Handles ties**
   It chooses the best status possible if we encounter a tie




```
// Most recent Time among reports for a given street
fun mostRecentTime[s: Street]: set Time {
  { t: reportsForStreet[s].createdAt |
    no t2: reportsForStreet[s].createdAt | t in t2.^next
  }
}

// Reports whose createdAt is the most recent Time (max
freshness)
fun mostRecentReports[s: Street]: set Report {
  { r: reportsForStreet[s] | r.createdAt in mostRecentTime[s] }
}
```

2

```
let rs = mostRecentReports[s] |

  // Case 1: no recent reports → keep current status
  (no rs) => s.status
  else (one rs) => rs.rStatus

  // Case 2: multiple recent reports → count statuses
  else {
    let optCnt = countReportsWithStatus[rs, Optimal] |
    let medCnt = countReportsWithStatus[rs, Mediocre] |
    let nmCnt  = countReportsWithStatus[rs, NeedsMaintenance] |

    (optCnt > medCnt and optCnt > nmCnt) => Optimal
    else (medCnt > optCnt and medCnt > nmCnt) => Mediocre
    else (nmCnt > optCnt and nmCnt > medCnt) => NeedsMaintenance
...
  }
```

3

```
else {
  (optCnt = medCnt and optCnt > nmCnt) => Optimal
}
```

```
else {
  Optimal
}
```

Executing "Run run$1 for 4 but 2 RegisteredUser, 3 BikePath, 3 Street, 2 Report, 3 Time, 1 Trip"
Actual scopes: 2 User, 2 RegisteredUser, 4 GeoPoint, exactly 3 Time, 3 Status, exactly 1 Optimal, exactly 1 Mediocre,
Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
4307 vars. 254 primary vars. 12765 clauses. 60ms.
Instance found. Predicate is consistent. 16ms.

GENERAL TESTING.

Executing "Run singleReportUpdates for 4"
Actual scopes: 4 User, 4 RegisteredUser, 4 GeoPoint, exactly 4 Time, 3 Status, exactly 1 Optimal,
Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
7762 vars. 432 primary vars. 21711 clauses. 67ms.
Instance found. Predicate is consistent. 21ms.

REPORT MECHANISM FULLY FUNCTIONAL .

Executing "Run freshnessWins for 4"
Actual scopes: 4 User, 4 RegisteredUser, 4 GeoPoint, exactly 4 Time, 3 Status, exactly 1 Optimal,
Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
8171 vars. 448 primary vars. 22629 clauses. 93ms.
Instance found. Predicate is consistent. 27ms.

FRESHNESS PREVAILS AMONG MANY REPORTS

Executing "Run majorityVotingOnConflict for 4"
Actual scopes: 4 User, 4 RegisteredUser, 4 GeoPoint, exactly 4 Time, 3 Status, exactly 1 Optimal,
Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
8148 vars. 448 primary vars. 22626 clauses. 50ms.
Instance found. Predicate is consistent. 13ms.

AMONG SAME FRESHNESS, MAJORITY WINS

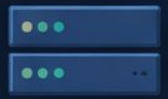
ARCHITECTURAL STYLES

Client-Server Architecture



Mobile App (Fat Client)

PRESENTATION LAYER

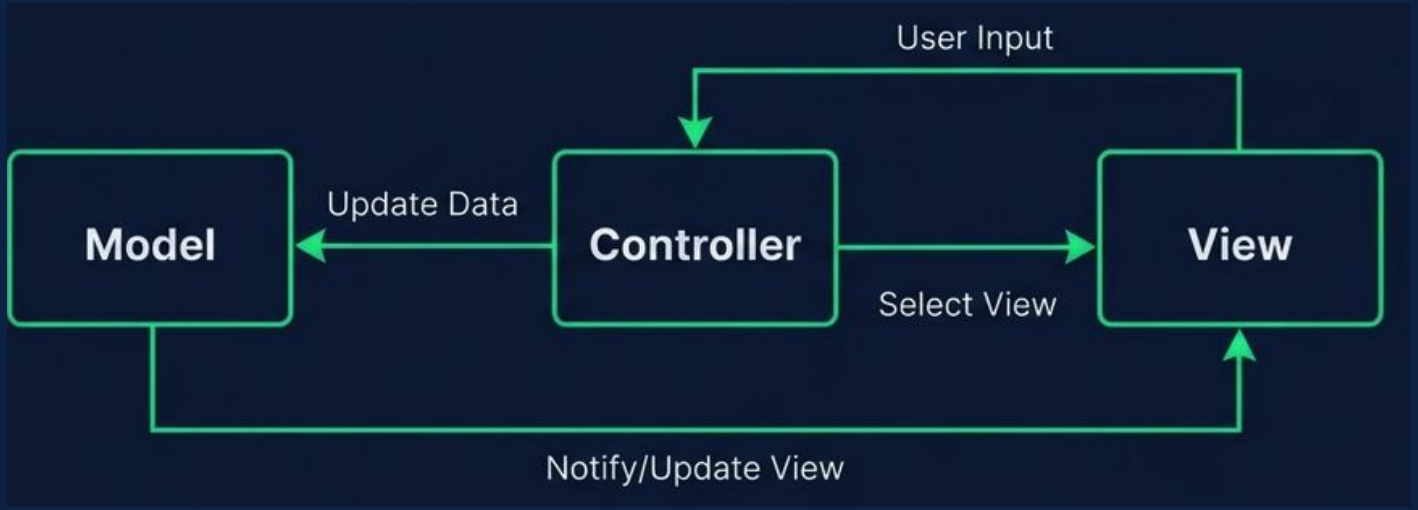


APPLICATION LAYER



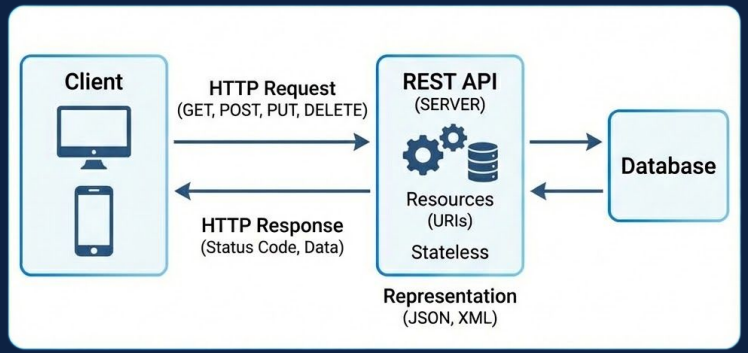
DATA LAYER

Model View Controller



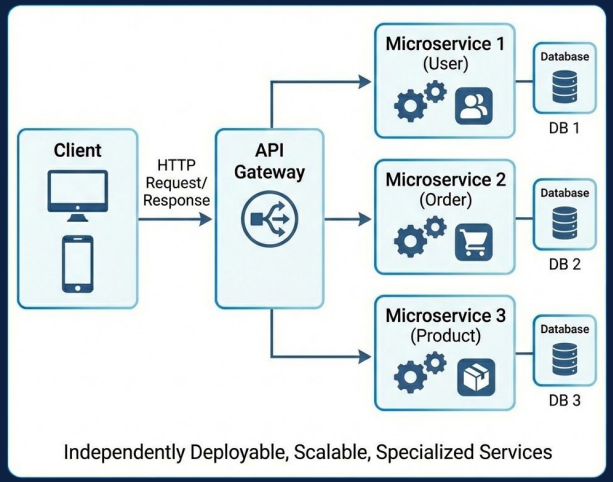
Other Patterns and Architectures

RESTful Architecture



- statelessness

Microservice Pattern



- Independent and decoupled components
- Load Balancing

COMPONENTS

CLIENT COMPONENTS

1. **Sensor Manager** Communication with sensors and data retrieving.
2. **Trip Detector** Automatic detection based on sensor data.
3. **Metrics Engine** Computation of trip metrics.
4. **Offline Sync Module** Offline re-synchronization during trip for sudden Internet disrupts.
5. **Connector Module** Bridge node between client and server components.

SERVER COMPONENTS

1. **User Manager** User management.
2. **Trip Manager** Intermediate node for trip recording.
3. **Bike Path Manager** Creation, validation and publication of Bike Paths.
4. **Path Recommendation System** Retrieval and ranking of searched Bike Paths.
5. **Report Manager** Street status and irregularities management.
6. **Street Manager** Updating streets' condition based on reports.
7. **External API Manager** Bridge node between server and external APIs.
8. **Data Access Module** Bridge node between server and DBMS.


INTERACTIONS

COMPONENT INTERACTIONS

CLIENT-SERVER

Client components and server components can’t interact directly with each other, the message stream is handled by the Connector Module.

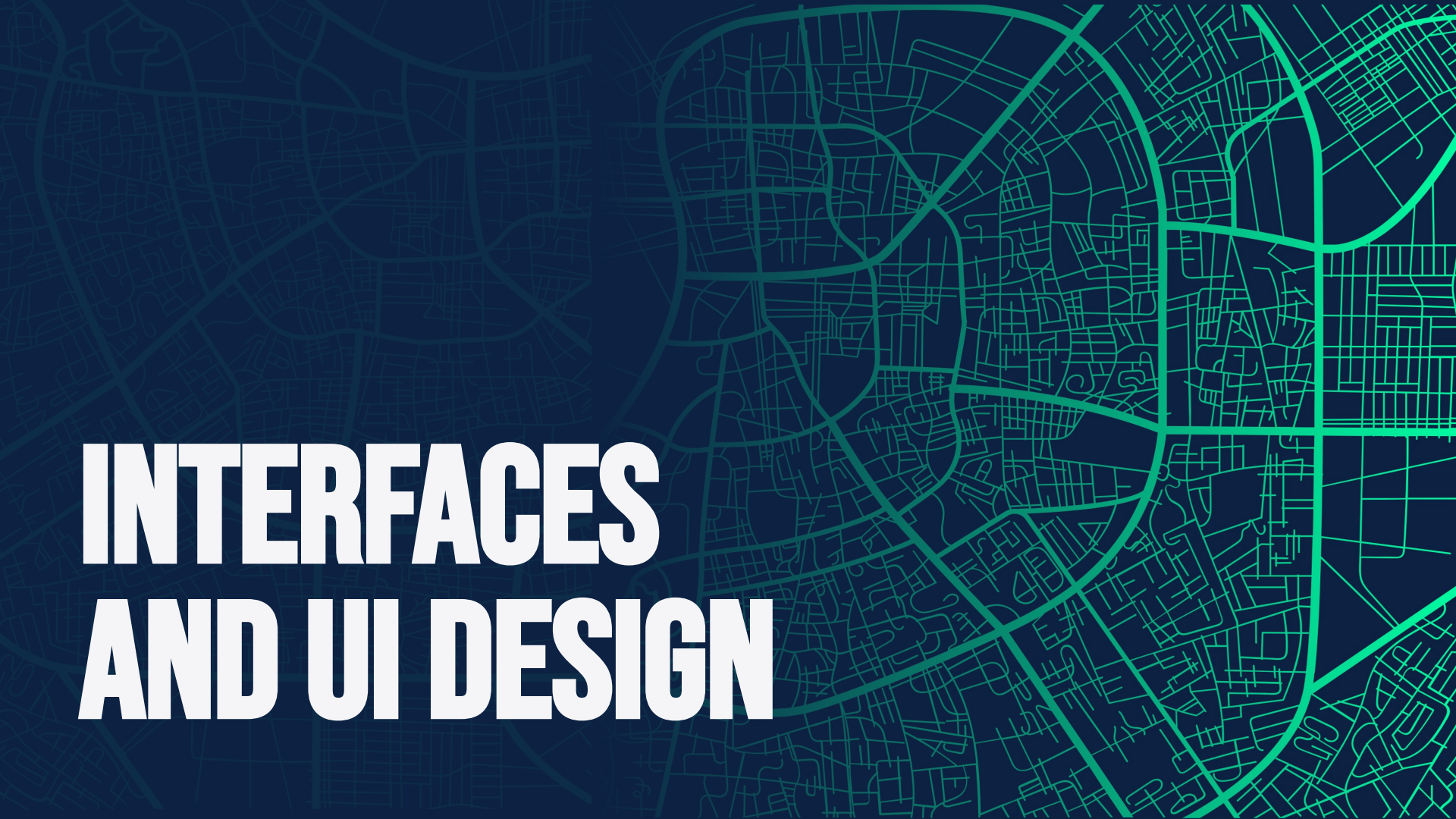
DATABASE

The data stream between server components and the database is handled by the Data Access Module, not allowing direct approaches.

EXTERNAL APIs

Weather and Navigation APIs are called by the dedicated component External API Manager, which gets and dispatches the received data.

Client components can interact with each other, so as the server components can do. Besides the Connector Module, no interactions between client and server are allowed.


INTERFACES
AND UI DESIGN

EVERY MAIN UI ACTION MAPS TO A SMALL, STABLE OPERATION.

S **`findsPath(origin: String, destination: String)`** in **Path Recommendation System**

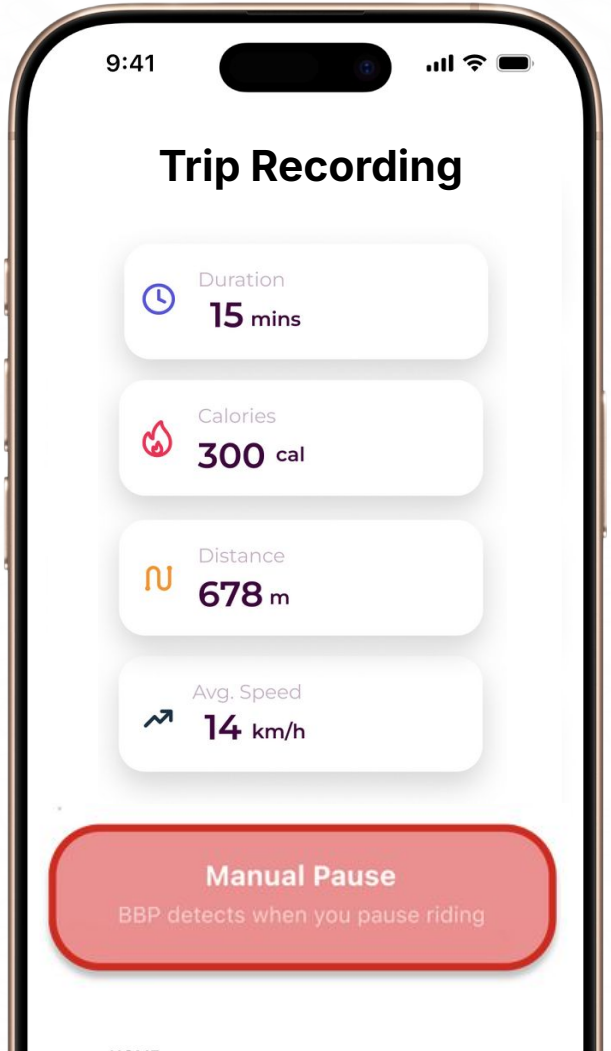
S **`createPath(pathData: Path)`** in **Bike Path Manager**

S **`updateStreetStatus(streets: list<Street>)`** in **Street Manager**

S **`checkStreet(street: Street)`** in **External API Manager**

C **`detectCycling()`** in **Trip Detector**

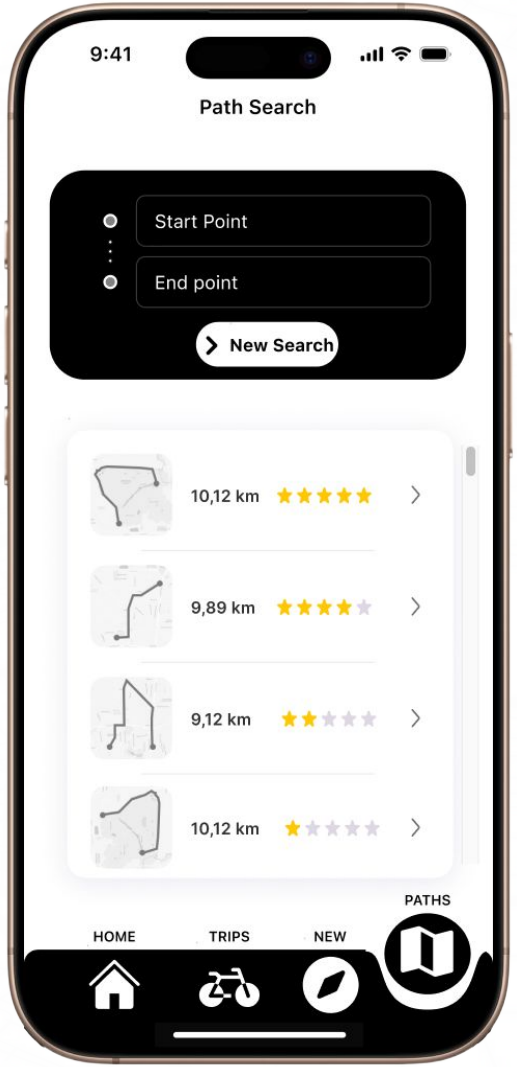
C **`updateMetrics(SensorData: SensorData)`** in **Metrics Engine**

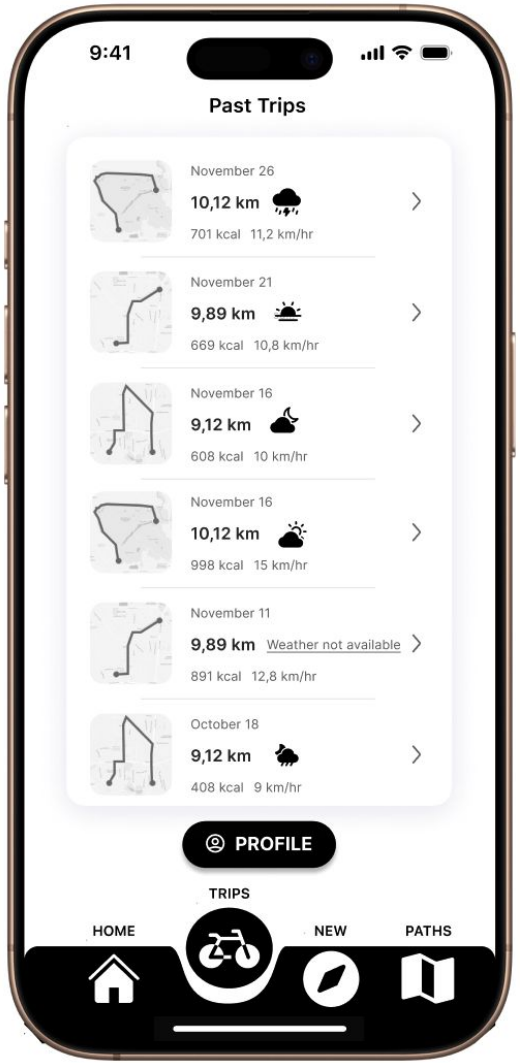


We intentionally adopt a simple interaction interface, in order to to **maximize learnability** and also **minimize cognitive load** during cycling, when attention is limited and directed to the road.

Following the **Human-Computer-Interaction** paradigm:

- During recording we **enhanced visibility** of system status with large, **low touch-precision** controls appropriate.
- UX-wise, we mitigate the possibility of sensor noise by ensuring **the user can always take control** (pause/stop).

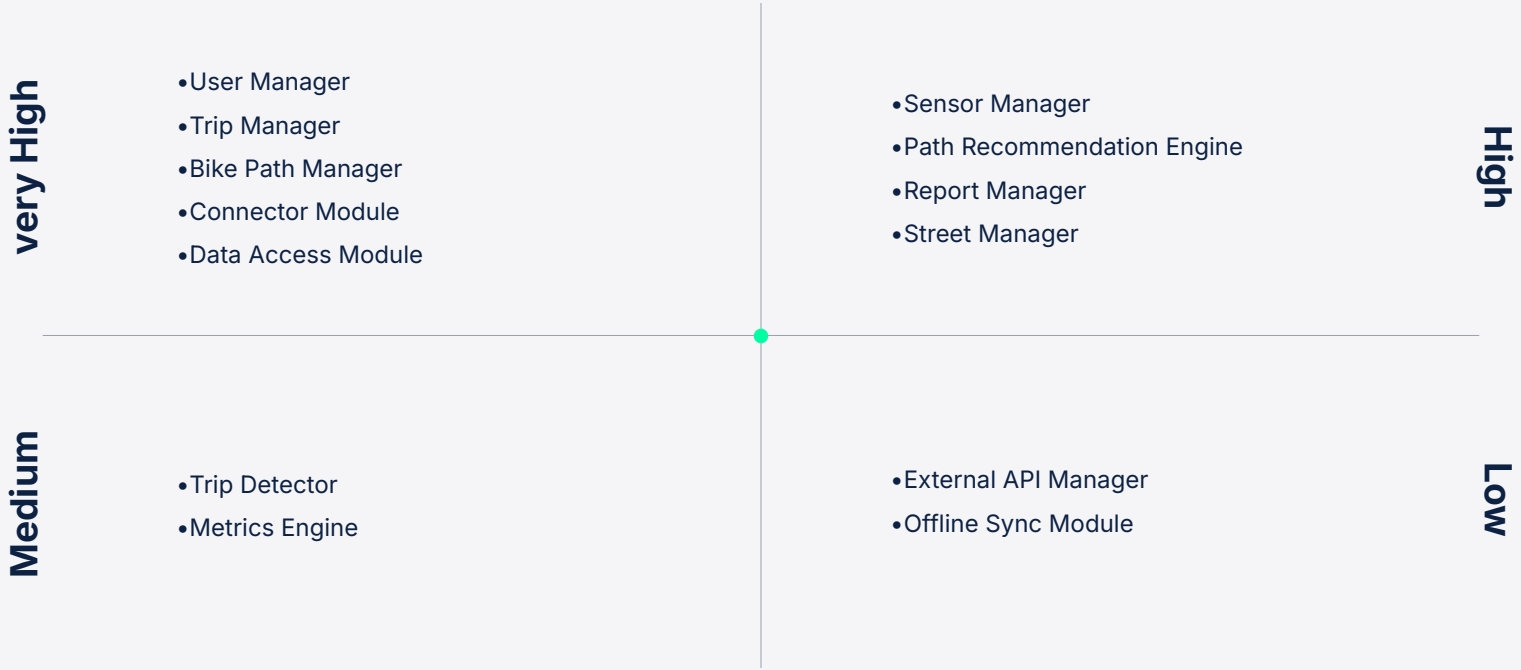

9:41
Path Search
Start Point
End point
New Search
10,12 km
9,89 km
9,12 km
10,12 km
HOME
TRIPS
NEW
PATHS

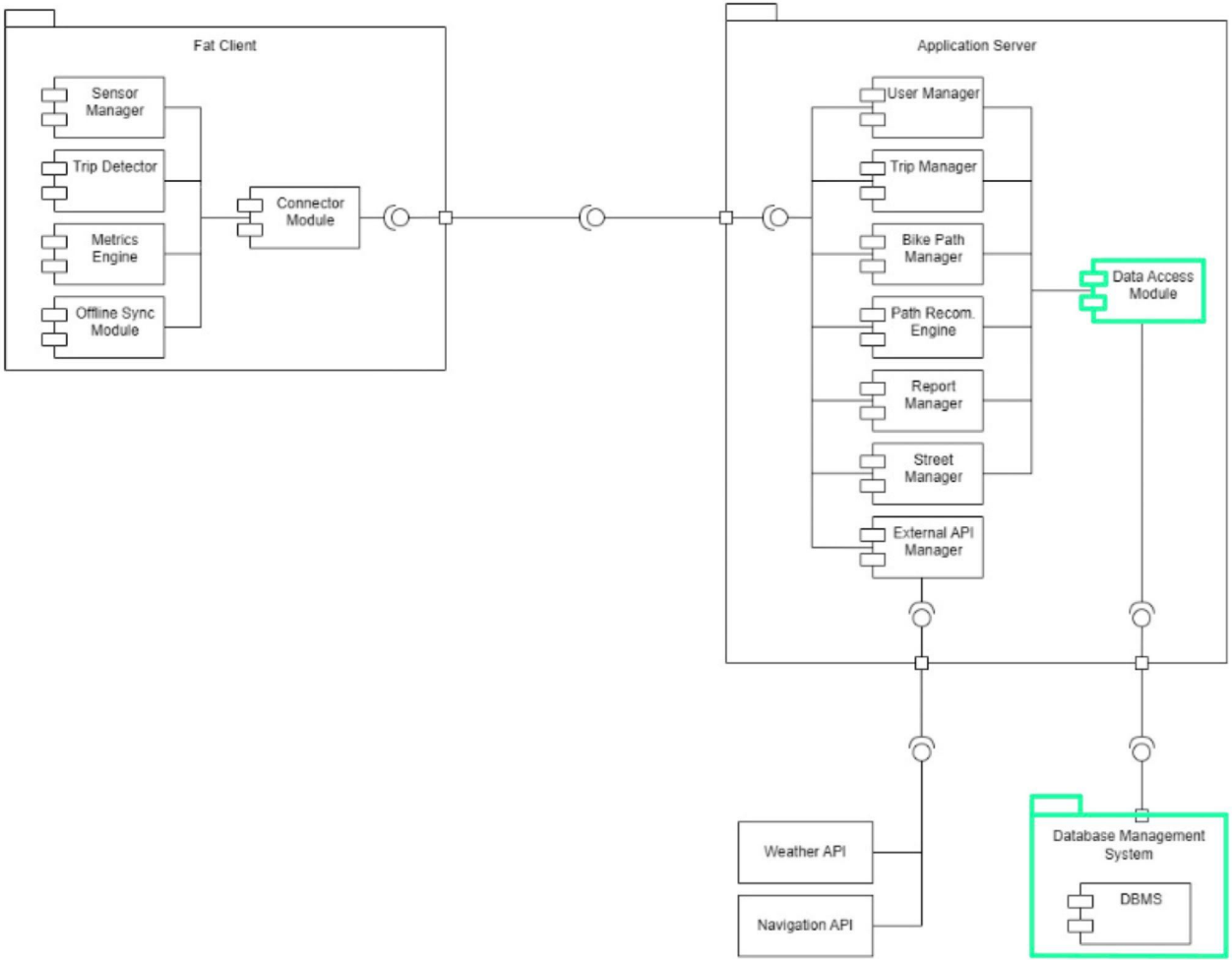

9:41
Past Trips
November 26
10,12 km
701 kcal 11,2 km/hr
November 21
9,89 km
669 kcal 10,8 km/hr
November 16
9,12 km
608 kcal 10 km/hr
November 16
10,12 km
998 kcal 15 km/hr
November 11
9,89 km
Weather not available
891 kcal 12,8 km/hr
October 18
9,12 km
408 kcal 9 km/hr
PROFILE
HOME
TRIPS
NEW
PATHS

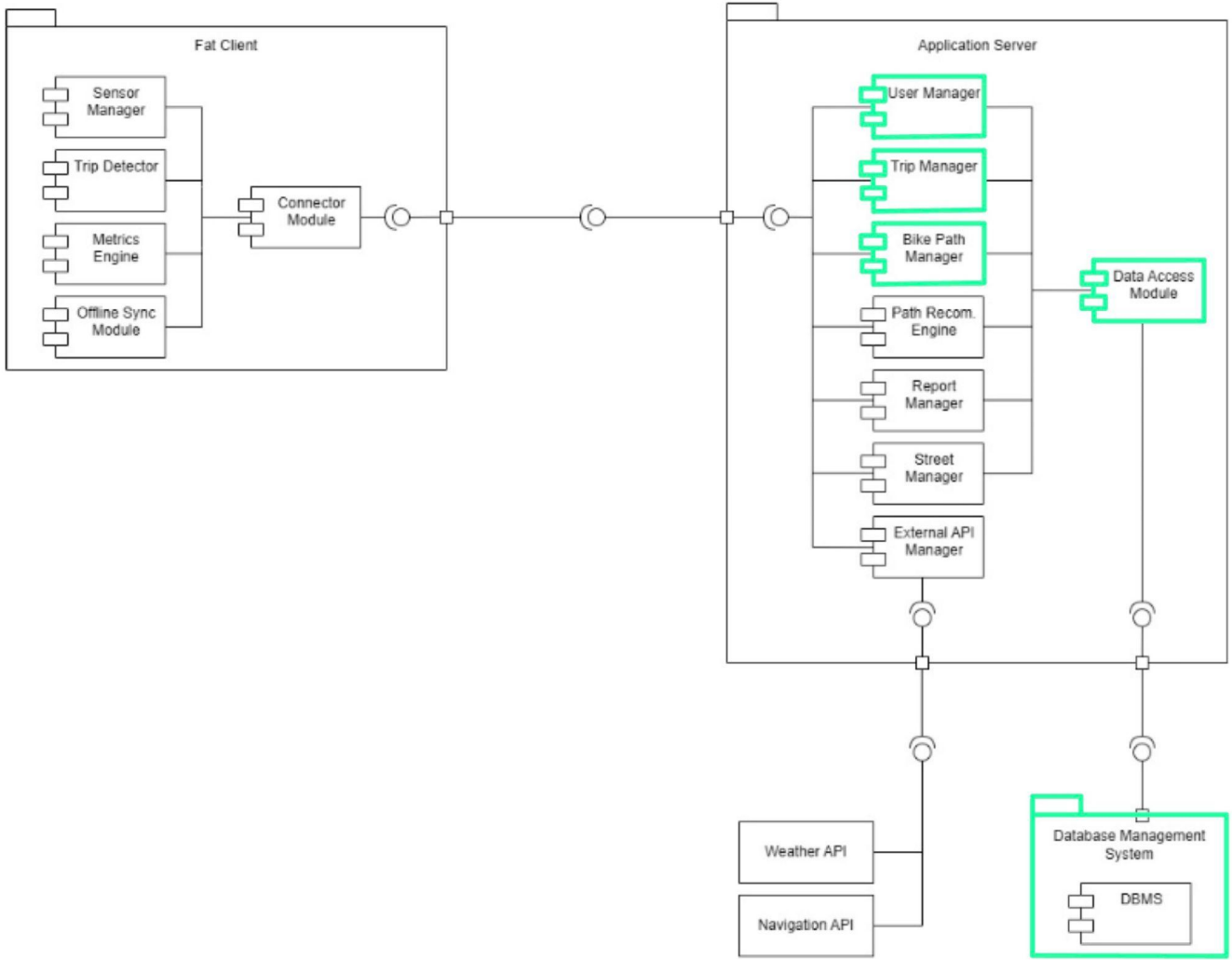

DEVELOPMENT
& TESTING

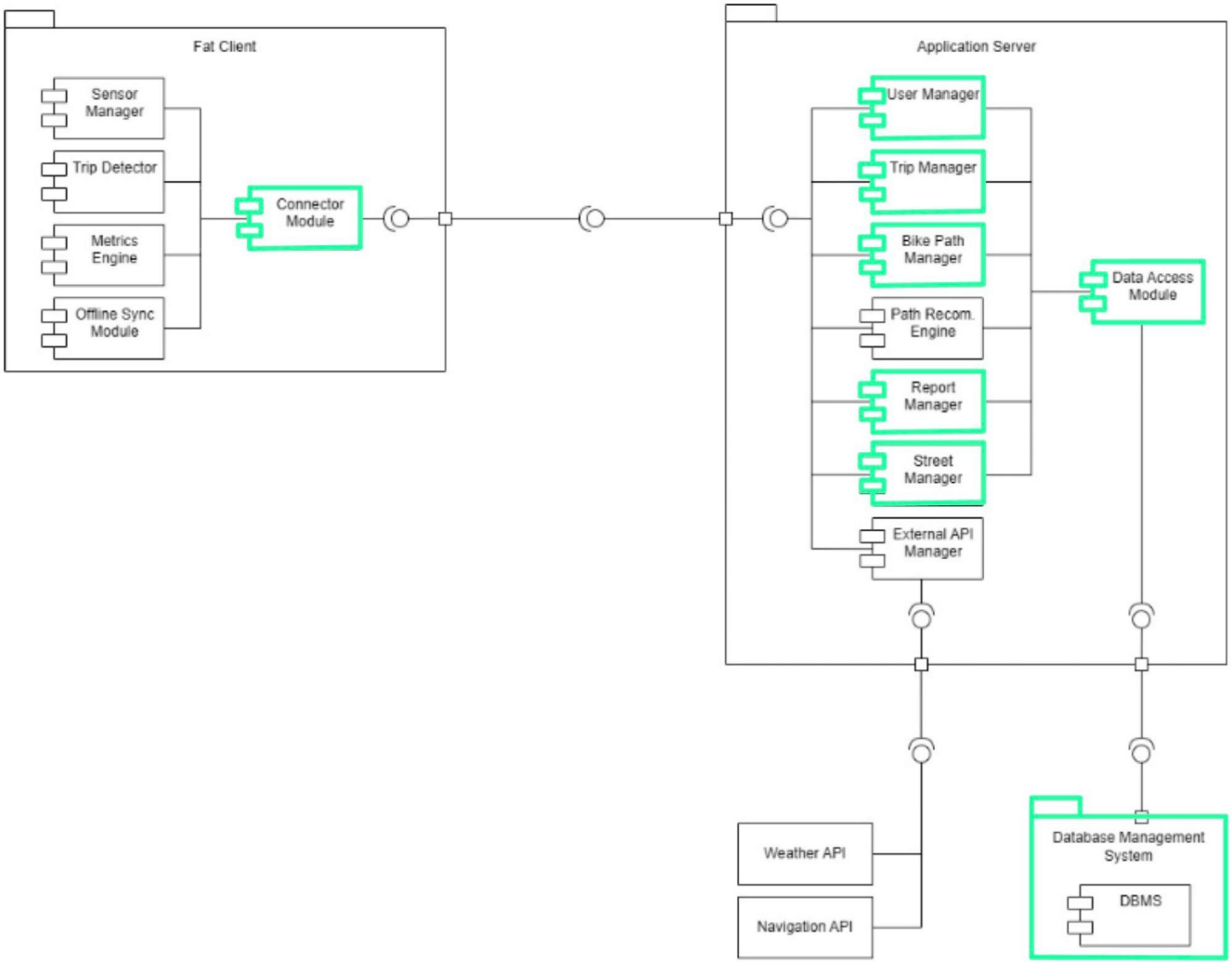
Component Priority

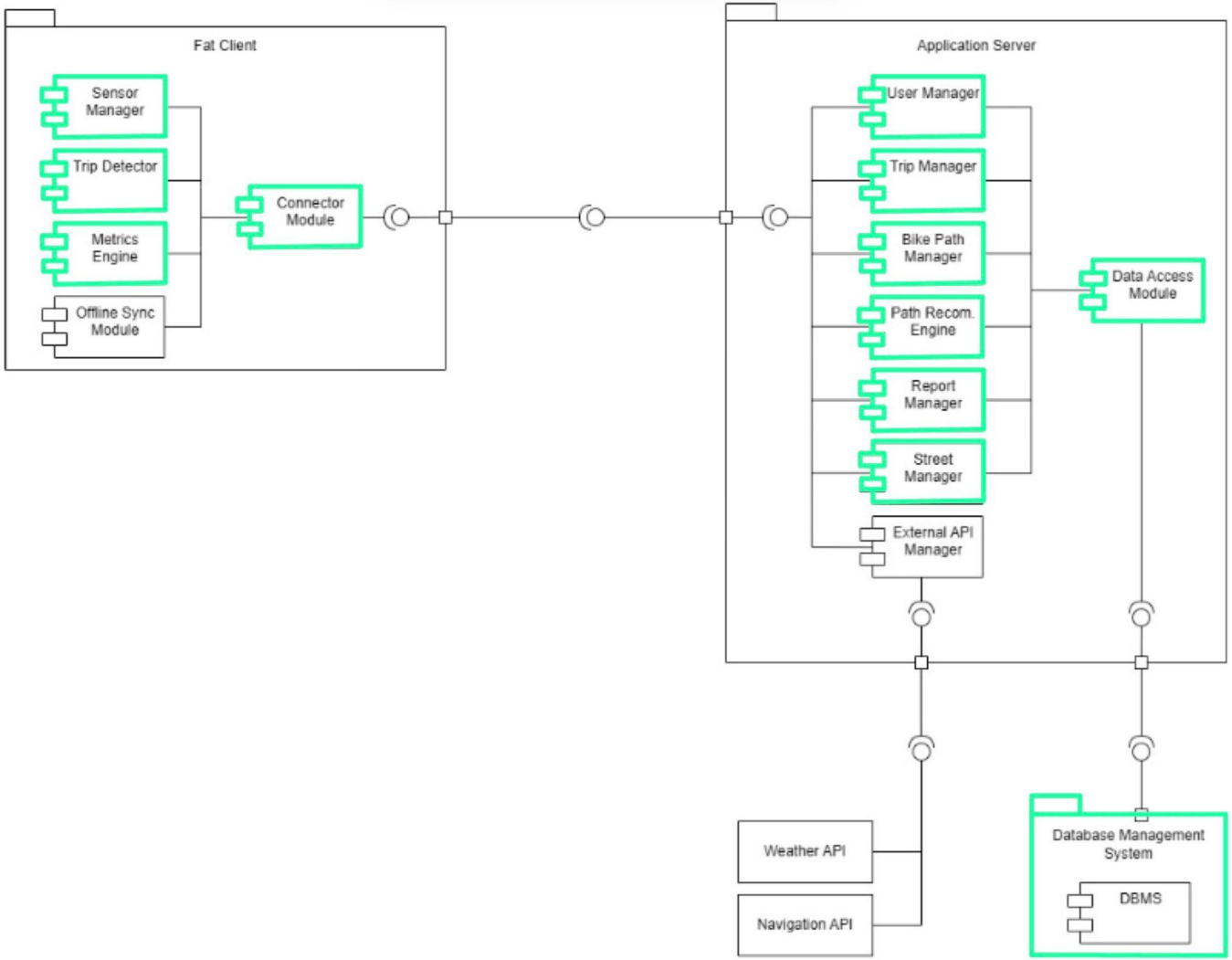
Which component to implement first?

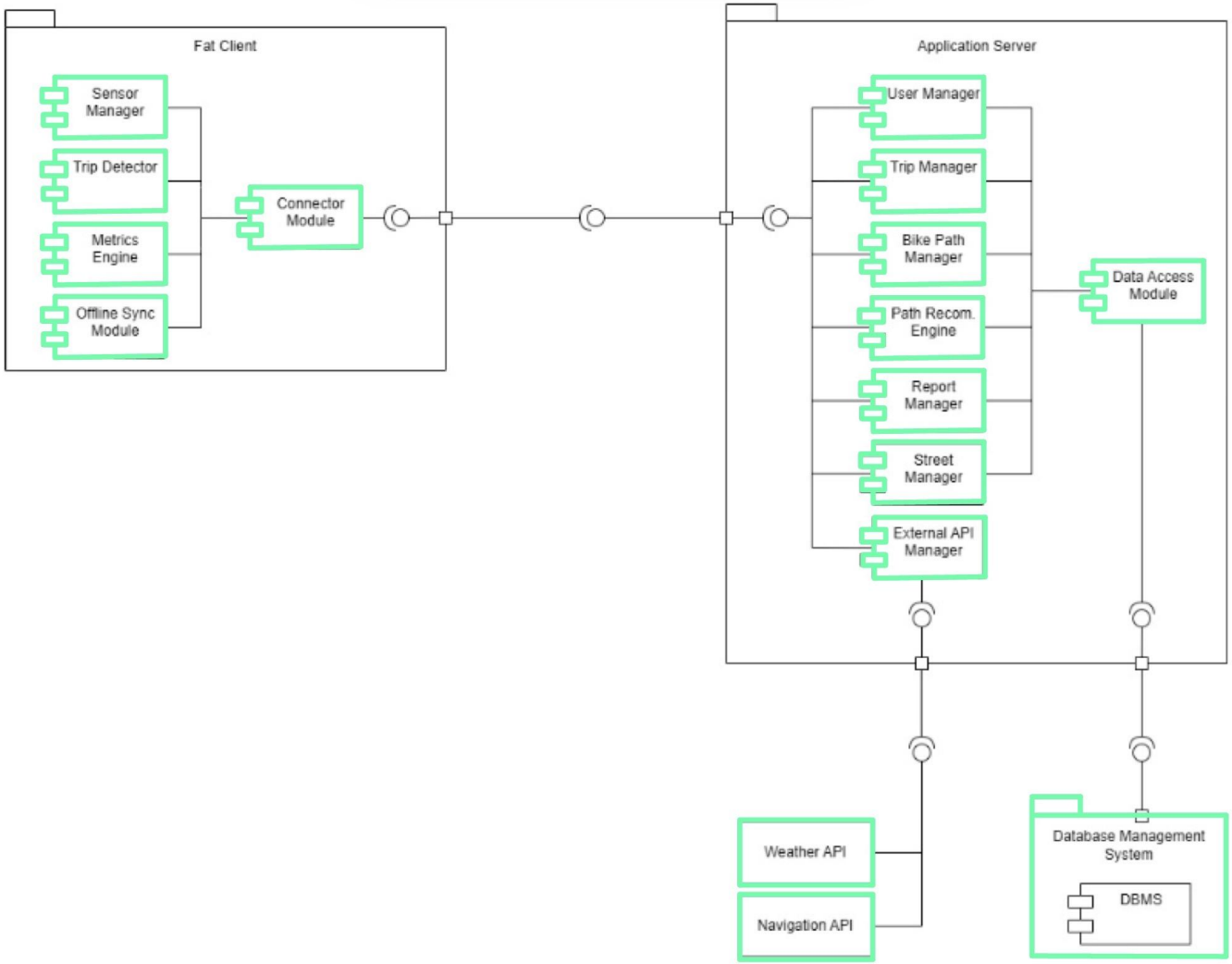



Fat Client
Sensor Manager
Trip Detector
Metrics Engine
Offline Sync Module
Connector Module
Application Server
User Manager
Trip Manager
Bike Path Manager
Path Recom. Engine
Report Manager
Street Manager
External API Manager
Data Access Module
Weather API
Navigation API
Database Management System
DBMS


Fat Client
Sensor Manager
Trip Detector
Metrics Engine
Offline Sync Module
Connector Module
Application Server
User Manager
Trip Manager
Bike Path Manager
Path Recom. Engine
Report Manager
Street Manager
External API Manager
Data Access Module
Weather API
Navigation API
Database Management System
DBMS


Fat Client
Sensor Manager
Trip Detector
Metrics Engine
Offline Sync Module
Connector Module
Application Server
User Manager
Trip Manager
Bike Path Manager
Path Recom. Engine
Report Manager
Street Manager
External API Manager
Data Access Module
Weather API
Navigation API
Database Management System
DBMS


Fat Client
Sensor Manager
Trip Detector
Metrics Engine
Offline Sync Module
Connector Module
Application Server
User Manager
Trip Manager
Bike Path Manager
Path Recom. Engine
Report Manager
Street Manager
External API Manager
Data Access Module
Weather API
Navigation API
Database Management System
DBMS


Fat Client
Sensor Manager
Trip Detector
Metrics Engine
Offline Sync Module
Connector Module
Application Server
User Manager
Trip Manager
Bike Path Manager
Path Recom. Engine
Report Manager
Street Manager
External API Manager
Data Access Module
Weather API
Navigation API
Database Management System
DBMS


THANKS FOR YOUR ATTENTION,
ANY QUESTIONS?